

Rethinking the Designing of Convolution Engine for Reconfigurable CNN Accelerator Using Sparse-Based Design Scheme

Yishuo Meng, Jianfei Wang^{id}, Siwei Xiang, Jia Hou, Zhijie Lin, Kuizhi Mei^{id}, *Member, IEEE*, and Chen Yang^{id}, *Member, IEEE*

Abstract—Convolutional neural networks (CNNs) are evolving as they are applied to more diverse environments and more difficult challenges. The evolving induces various convolution modes (e.g., 1×1 convolution, 2-stride convolution and rectangle convolution) in current CNNs and makes it difficult for the hardware accelerators to efficiently support such various convolution modes. In this paper, it is found that an important difference of these convolution modes is the computation density. Therefore, the above convolution modes are regarded as structured sparse and claims that sparse-based design methodology can be applied for the implementation of the reconfigurable CNN accelerator. Subsequently, two critical architectural parameters, including input tile size and convolution engine (CE) scale, are evaluated based on Standard deviation of calculations (SDC), unsupported convolution mode (UCM) and unsuitable IFM size (UIS), DSP utilization ratio (DUR) as well as hardware resource overhead (HRO), respectively. With the aid of the optimal parameters, a high-parallelism and flexible CE array and a high-performance and reconfigurable CNN architecture are designed. The accelerator was implemented on a Xilinx VC709 FPGA and ran at a clock frequency of 300 MHz, achieving 921.60 to 1382.40 GOPS while supporting various convolution modes. Compared with previous dense-/sparse-based works, the proposed accelerator can realize $1.35\times$ to $10.77\times$ improvements on performance and $1.22\times$ to $2.84\times$ improvements on DSP efficiency while deploying VGG16.

Index Terms—Convolutional neural network, reconfigurable accelerator, design space exploration, DSP efficiency.

I. INTRODUCTION

STATE-OF-THE-ART convolutional neural networks (CNNs) have shown superior performance in various fields, such as image classification [1], self-driving cars [2] and semantic segmentation [3]. Unluckily, the remarkable performance of CNNs is achieved at the cost of massive parameters and computations. Consequently, the development of customized hardware accelerators has emerged as a promising and rapidly growing field. However, CNNs are continuously evolving to adapt to diverse environments

Received 23 February 2024; revised 29 May 2024 and 5 November 2024; accepted 14 March 2025. Date of publication 8 April 2025; date of current version 30 July 2025. This work was supported by the National Natural Science Foundation of China under Grant 62176206. This article was recommended by Associate Editor X. Zhang. (*Corresponding author: Chen Yang.*)

The authors are with the School of Microelectronics, Xi'an Jiaotong University, Xi'an, Shaanxi 710049, China (e-mail: 3121156027@stu.xjtu.edu.cn; jfwang@stu.xjtu.edu.cn; X1004312107@stu.xjtu.edu.cn; hjwst0314@stu.xjtu.edu.cn; linzhijie@stu.xjtu.edu.cn; meikuizhi@mail.xjtu.edu.cn; chyang00@xjtu.edu.cn).

Digital Object Identifier 10.1109/TCSI.2025.3554332

TABLE I

THE CONVOLUTION MODES OF CURRENT CNNs

CNN	Convolution mode (Filter size/Stride)
ResNet-18	$3 \times 3/1$ $3 \times 3/2$ $1 \times 1/2$ $7 \times 7/2$
VGG16	$3 \times 3/1$
GoogleNet	$1 \times 1/1$ $3 \times 3/1$ $5 \times 5/1$ $7 \times 7/2$
MobileNetV2	$1 \times 1/1$ $3 \times 3/1$ $3 \times 3/2$
ResNet-50	$1 \times 1/1$ $1 \times 1/2$ $3 \times 3/1$ $3 \times 3/2$ $7 \times 7/2$
InceptionV4	$1 \times 1/1$ $3 \times 3/1$ $3 \times 3/2$ $1 \times 3/1$ $3 \times 1/1$ $1 \times 7/1$ $7 \times 1/1$
YOLOV3	$1 \times 1/1$ $3 \times 3/1$ $3 \times 3/2$
YOLOV3-tiny	$1 \times 1/1$ $3 \times 3/1$

and increasingly challenging tasks. A significant indicator of this evolution lies in the convolution modes (e.g., filter size and stride). The first widely recognized large-scale CNN is referred to as AlexNet [4], which employs specific convolution modes, such as 11×11 convolution with a stride of 4. Subsequently, a structured CNN named VGG16 applies only one convolution mode (i.e., 3×3 convolution) and achieves outperformed accuracy [5]. 1×1 convolution is utilized in GoogleNet [6] for parameter reduction and information combination across channels. 2-stride convolution, depth wise separable convolution and rectangle convolution (e.g., 1×3 convolution) are further applied in the following CNNs [7], [8], [9], [10], [11]. Consequently, the aforementioned CNN evolving makes it difficult for the hardware accelerators to efficiently support such a wide range of convolution modes.

Specifically, the convolution modes in current CNNs are presented in TABLE I. It is observed that the evolution of CNNs has not led to convergence for the convolution modes. As demonstrated in TABLE I, there are six types of filter sizes and two types of strides are still employed in current CNNs, resulting in a diverse range of convolution modes. Moreover, this diversity poses challenges for flexibly and efficiently accelerating these CNNs. To solve the above problem, a filter decomposition method is proposed and applied to design reconfigurable CNN accelerators [12], [13], [14], [15], [16], [17]. By applying this approach, the filters with various size and stride are split into multiple filter blocks with uniform size and stride, thus enhancing the flexibility and efficiency of reconfigurable CNN accelerators while using a unified acceleration unit. However, this method would induce nonnegligible zero elements while splitting the filters whose sizes are not divisible by the filter block size. Moreover,

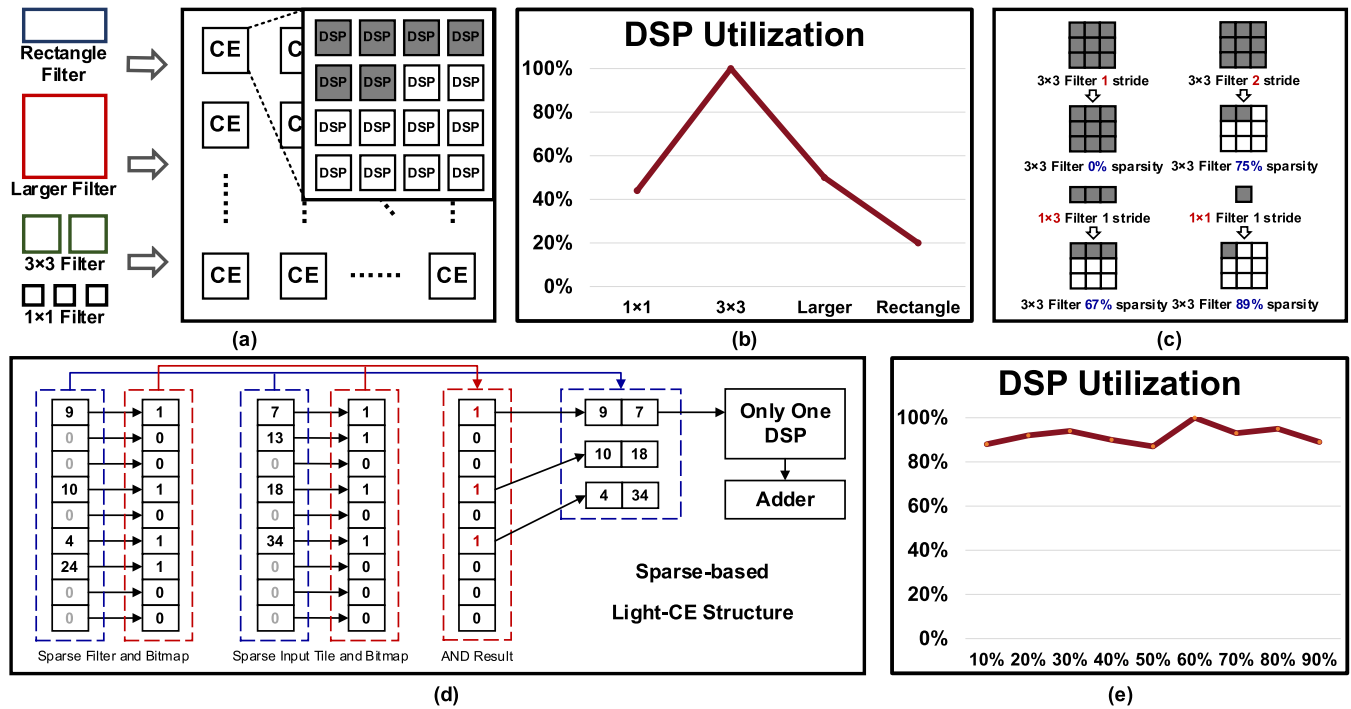


Fig. 1. The motivation of this work. (a) The inefficient CE while facing different types of filters. (b) The DSP utilization of different types of filters. (c) The equivalent sparsity taking 3×3 filters as a reference. (d) The light-CE designing scheme applied in work [19]. (e) The DSP utilization under different sparsity.

this method is only applicable for square convolution (i.e., $n \times n$ convolution), thereby excluding support for rectangular convolutions such as 1×3 or 1×7 convolution. In this paper, it is discovered that a significant difference between these convolution modes is the computation density, which leads to different number of valid computations in the split input tiles while using tile-based dataflow. Therefore, the variety of valid computations negatively impacts the digital signal processor (DSP) utilization in the convolution engine (CE), thereby limiting the performance of reconfigurable CNN accelerators. Fortunately, this issue (i.e., the variety of valid computations) also occurs in sparse CNNs. CNN pruning preserves uncertain number of non-zero weights in different filters, which also results in different computation density. To address this problem, a light-CE with reduced multiply accumulate (MAC) units has been proposed to keep the CE busy and thereby enhance the DSP efficiency of sparse-based accelerators [18], [19]. Specifically, as shown in Fig. 1(d), in work [19], there is only one MAC unit in a CE to operate one filter. With the aid of light-CE, inefficient parallel computing is transformed into efficient serial computing, thereby significantly increasing the DSP efficiency.

Consequently, in this paper, we first attempt to regard the various convolution modes as structured sparse and apply the design methodology of the light-CE to the implementation of reconfigurable CNN accelerator. Subsequently, based on this thought, multiple critical parameters of the designed accelerator, including the input tile size and the CE scale, are determined through the design space exploration. Finally, we implement a reconfigurable CNN architecture with a high-throughput and flexible CE array using these determined parameters and compare it with the previous reconfigurable CNN accelerators. The main contributions of this paper are as follows:

A. Design Methodology Determination

Unlike other reconfigurable CNN architectures, we firstly analyze the difference of the convolution modes and then extract the critical similarity between these modes and sparse CNNs. We subsequently adopt a design methodology inspired by sparse-based architecture (i.e., light-CE) for the following design space exploration and hardware architecture implementation.

B. Design Space Exploration

After establishing the design principle, we evaluate the critical parameters of the reconfigurable architecture through design space exploration. First, we evaluate the input tile size, which could reflect the read/write bandwidth, based on the following factors: the standard deviation of computations (SDC), the number of unsupported convolution mode (UCM) and unsuitable IFM (i.e., input feature map) size (UIS). Afterwards, we select the scale of CE, which could impact the actual performance and DSP efficiency, based on the DSP utilization ratio (DUR) and hardware resource overhead (HRO).

C. Hardware Architecture Implementation

With the aid of the chosen design parameters, we design a high-parallelism and flexible CE array along with a four-stage accumulator. Moreover, we implement a reconfigurable CNN architecture at a runtime frequency of 300 MHz, which could support all of the listed convolution modes in TABLE I and maintain 94.7% DSP utilization in average, providing flexible and efficient acceleration for the evolving CNNs.

The novelty of the proposed architecture is mainly reflected in the following two points: (1) It is found that the main difference of multiple convolution modes lies in their diverse computation density. Therefore, we build a novel sparse-based

method to analyze the convolution modes and use light-CEs in the implementation of reconfigurable CNN architecture. However, different from the previous sparse-based works [18], [19] that utilize pre-defined scale of light-CEs, the architectural parameters are further evaluated through design space exploration to provide a wider range of convolution modes and IFMs. (2) Different from the previous reconfigurable CNN accelerators that utilize heavy CE, light-CEs with an optimal CE scale is applied in this study to achieve enhanced DSP utilization and hardware overhead. In the meanwhile, the increased input/output computational parallelism enables that our designed architecture can realize similar and even higher actual performance while compared with previous reconfigurable CNN accelerators.

II. RELATED WORKS AND MOTIVATION

In this section, recent works on the evolution of CNNs [4], [5], [6], [8], [9], [20], [21], [22], [23], [24], [25], [26] and the development of CNN accelerators [12], [13], [14], [15], [16], [17], [28], [29], [30], [31], [32] are discussed. Then, the motivations of the proposed reconfigurable CE and hardware architecture is introduced in detail.

A. Related Works

1) *The Evolution of CNNs*: LeCun et al. proposed a classic and efficient CNN, LeNet-5, which is one of the origins of neural networks [20]. AlexNet and VGG-16 were then proposed by Krizhevsky et al. and Simonyan et al., respectively. The above two CNNs which hold totally different structures achieved significant success in many vision-based fields [4], [5]. As CNNs are employed to increasingly complicated tasks and placed for more stringent demands, CNNs are constantly evolving. In GoogleNet, multiple branches including different convolution types and pooling were incorporated in a layer, which is named inception structure. Moreover, 1×1 convolution was also applied for parameter reduction [6]. 2-stride convolution was utilized in MobileNetV2 to replace pooling layer [9]. Subsequently, different from the previous CNNs, which always use square convolutions, rectangle convolution was applied in InceptionV4 to further reduce the multiplications [8]. The evolving of CNNs realize significant accuracy improvement while markedly reducing the parameters and computations. On the other hand, network pruning [21], [22], [23], [24], [25], [26], including structured pruning [21], [22], [23] and unstructured pruning [24], [25], [26], can also reduce the parameters of original CNNs to alleviate overly intensive operations. A large amount of evidence has shown that the above works achieved excellent success in accelerating CNN inference.

2) *The Development of CNN Accelerators*: Customizing hardware accelerators is a promising way for CNN acceleration. For example, work [27] designed a dynamic reconfigurable accelerator for LeNet. Subsequently, both of works [28], [29] can support AlexNet and VGG-16, achieving better flexibility and efficiency. Facing the massive calculations of popular CNNs, deploying convolution acceleration algorithms, including Winograd [12], [13], [30] and FFT [30], [31], [32] to customize hardware accelerators can efficiently improve the performance of existing methods. However, as the evolving of CNNs (especially the application of 1×1 and 2-stride convolutions), the changing convolution modes and network structures make it difficult for the deployment of traditional convolution acceleration algorithms. Reconfigurable

oriented design schemes and corresponding architecture can break down the barriers between changing convolution modes and fixed hardware structures, thereby achieving excellent compatibility and outperformed performance [12], [13], [14], [15], [16], [17]. For instance, filter decomposition scheme, which was proposed in work [12] and applied in works [13], [14], [15], [16], [17], can divide the filters with multiple sizes and strides to 3×3 filter blocks with 1-stride. Therefore, based on the filter decomposition method, corresponding reconfigurable architectures with unified computation units are designed and implemented, achieving improved reconfigurability and performance. As CNN continues to evolve, the appearance of novel convolution modes makes the designing for reconfigurable architecture a necessary way for CNN acceleration.

B. Motivation

The evolving of CNNs makes it challenging for the deployment of current evolving CNNs as well as the implementation of reconfigurable CNN accelerators.

First, the evolving of CNNs has led to an expansion in the convolution library utilized in current CNNs, thereby imposing more stringent compatibility and efficiency requirements for CNN accelerators. Specifically, as for the filter size, 3×3 convolution used to be the mainstream convolution in classical CNNs [4], [5]. Subsequently, 1×1 convolution has gained popularity in current lightweight CNNs such as ResNet-50 and MobileNetV2 [7], [9]. Moreover, larger filters still play an important role in evolved CNNs. For instance, GoogleNet employs 5×5 convolution in each inception module [6]. In ResNet-50, 7×7 convolution is applied in the first layer to help increase the receptive field [7]. Additionally, although 1-stride convolution is the most commonly used, 2-stride convolution is also utilized as a replacement for pooling layers in current CNNs to reduce the IFM sizes [7], [8], [9]. In addition to filter size and stride, there exists a diverse range of convolution types including square convolution (i.e., 3×3 convolution), rectangle convolution (e.g., 1×3 convolution) and dilated convolution [33]. Unfortunately, it is worth noting that the evolving of CNNs don't make convergence in terms of filter sizes, strides and convolution types. Therefore, the variability in the above factors poses challenges when deploying evolving CNNs on CNN accelerators.

Aiming at the aforementioned issue, filter decomposition scheme is one of the popular algorithms that can be applied to reconfigurable CNN acceleration [13], [14], [15]. In particular, in work [13], a stride-based filter decomposition method is introduced and applied for unified Winograd acceleration, achieving significant improvement on the flexibility and efficiency of CNN accelerators. Furthermore, this method also contributes to sparse-based hardware implementation and mixed-bit CNN acceleration [14], [15]. Unluckily, it should be noted that this approach has certain limitations. First, when encountering situations where the original filter size cannot be evenly divided by the decomposed filter size $\times n$, significant invalid zero elements are introduced. For example, in work [13], 44% invalid zero elements would be padded to 5×5 filters. When considering an original filter size of 7×7 , the percentage of invalid elements increases to 65%. The invalid elements would also incur unnecessary calculations, therefore achieving compatibility at the cost of efficiency. Moreover, the above filter decomposition method is

TABLE II
THE CONVOLUTION DENSITY AND EQUIVALENT SPARSITY FOR
DIFFERENT CONVOLUTIONAL MODES

IFM	Input/Output Parallelism	Filter size/Stride	Total MACs	Computation density	Density (Nor.)	Sparsity (Eq.)
28×28	64/64	3×3/1	28901376	576	1.00	0%
		1×3/1	9633792	192	0.33	67%
		3×3/2	7225344	144	0.25	75%
		1×1/1	3211264	64	0.11	89%

not suitable for the novel convolution types including rectangle convolution and dilated convolution.

Third, after analyzing the convolution modes, it is found that the core difference among them lies in their computation density, which refers to the number of multiplications performed on each input activation. TABLE II exhibits the computation density of four typical convolution modes. IFM size, input and output parallelism are predefined to 28, 64 and 64, respectively. It is observed that the computation density of 3×3 convolution with 1-stride, 1×3 convolution with 1-stride, 3×3 convolution with 2-stride, 1×1 convolution with 1-stride is 576, 192, 144 and 64, respectively. Furthermore, the normalization of computation density for the above computation modes is also evaluated and presented. That means the total computations in an input tile could be various among different convolution modes while deploying tile-based dataflow, which is the common-sense dataflow used in current CNN accelerators. Consequently, the CE array with a fixed number of DSPs of current reconfigurable CNN accelerators (Fig. 1(a)) would experience inadequate DSP utilization and unstable computation efficiency, as depicted in Fig. 1(b).

This issue is also faced and addressed in sparse CNN acceleration works. It has been demonstrated that a significant number of weights in CNNs are unimportant, and setting them to zero only leads to negligible accuracy loss [21]. Therefore, CNN pruning has emerged as an effective way to reduce the computational burden of CNNs, thereby obtaining sparse CNNs. In sparse CNNs, the number of nonzero weights of each filter is uncertain, resulting in varying computation densities. Therefore, we aim to convert the different convolution modes into distinct levels of sparsity in TABLE II and subsequently leverage the design methodology for sparse CNN accelerations in order to implement reconfigurable CNN acceleration. As illustrated in TABLE II and Fig. 1(c), first, 3×3 convolution with 1-stride can be regarded as a dense filter. Similarly, 1×3 convolution with 1-stride, 3×3 convolution with 2-stride, 1×1 convolution with 1-stride can also be considered as 67%, 75% and 89% sparsity. Consequently, as shown in Fig. 1(d), a light-CE with reduced DSPs is designed to serially operate the convolutions [19], therefore, the various computations are operated in different clock cycles, thereby achieving adequate DSP utilization and stable computation efficiency, as shown in Fig. 1(e).

Motivated by the aforementioned challenges, first, viewing various convolution modes from the perspective of sparsity (filter size unification), we apply a unified-tile based dataflow and light-CEs for the implementation of reconfigurable hardware accelerator. Additionally, we firstly operate the design space exploration for tile size. Based on the optimal tile size, CE scale is further evaluated based on the DSP utilization and hardware overhead. Finally, we implement a novel reconfigurable CNN acceleration design using the evaluated optimal architectural parameters. The evaluation results demonstrate

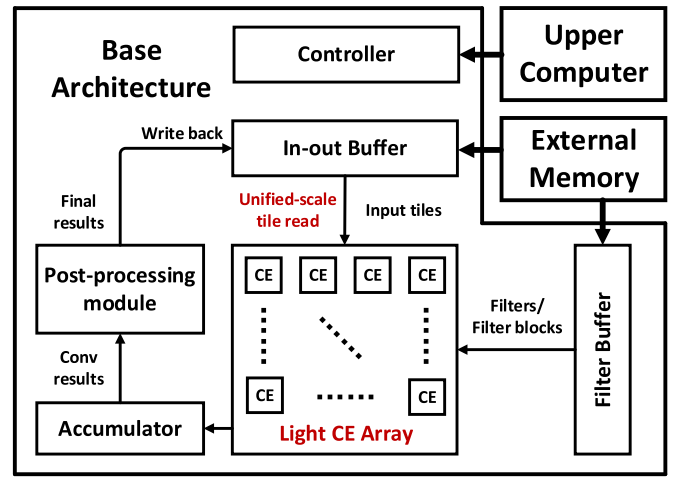


Fig. 2. Base architecture applied in the design space exploration.

that our proposed architecture can achieve 80.1% average DSP utilization and 94.7% weighted DSP utilization. Furthermore, it is no need to consider the load imbalance problem in our research, this is because that the equivalent structured sparsity results in the same workload at different input parallelism.

III. THE INTRODUCTION OF DESIGN SPACE EXPLORATION

Before the operating design space exploration, it is necessary to introduce the methodology of design space exploration proposed in this work. Firstly, the base architecture is shown in Fig. 2. The base architecture includes controller, in-out/filter buffer, light-CE array, accumulator and post-processing module. Actually, some parts of this base architecture derive from the generalization of some classical CNN accelerators [12], [13], [18], [54], [55], thereby the base architecture is relatively simple. However, this is because we try to conclude the current reconfigurable CNN accelerators and build a generally-used architecture model. Therefore, the subsequent design space exploration and critical parameter determination can be adapted to more works. Moreover, there are also some our designed highlights in this architecture, which are derived from our previous works [12], [13], [15], [36]. Specifically, in terms of the access for IFMs, a unified-scale tile-based dataflow (the split input tile size remains unchanged while splitting different IFMs among CNN layers) is applied and a simplified read-write strategy is achieved in the architecture, which is rarely presented in the previous reconfigurable CNN accelerators. Additionally, in the methodology, we conclude the indexes which are applied for architecture evaluation and comparison, including performance, DSP efficiency and hardware resource overhead (e.g., LUT, DSP and BRAM). Furthermore, based on the extracted indexes, the critical architectural parameters that can significantly affect the above indexes are analyzed. And the input tile size and CE scale are selected in the methodology as the evaluated parameters. Finally, five metrics including the adaptability for filters/IFMs, DSP utilization and hardware overhead are evaluated under different input tile sizes and CE scales. Consequently, the optimal tile size and CE scale are chosen to implement a reconfigurable architecture to realize the excellent performance and reduced hardware overhead.

The computation methods for the proposed metrics are presented in TABLE III. Specifically, standard deviation of calculations (SDC) represents that the deviation between the

TABLE III
THE COMPUTATION METHOD FOR THE PROPOSED METRICS

Metric	Computation method
SDC	Theoretical analysis
UCM	Theoretical analysis
UIS	Theoretical analysis
DUR	Architecture modeling and theoretical analysis
HRO	Measurement on specific hardware

maximum and minimum number of computations in different convolution modes for a unique input tile size, thereby it can be directly computed according to the size of the filters and input tiles, which is based on theoretical analysis. In addition, the number of unsupported convolution mode (UCM) and the number of unsuitable IFM size (UIS) reflect the number of convolution modes that cannot be supported by each input tile size and the number of IFM size that is not suitable to the selected input tile size, therefore, the above two metrics (i.e., UCM and UIS) are also evaluated based on theoretical analysis according to the filter/IFM size applied in CNNs and input tile size. Moreover, the DSP utilization ratio (DUR) is obtained by deploying different convolution modes to multiple architectures with specific CE scales. Consequently, the DUR is computed based on the combination of theoretical analysis and architecture modeling. Finally, the hardware resource overhead (HRO) is evaluated based on the synthesis and implementation on VC709. Therefore, HRO is computed based on actual measurement. To provide a more comprehensive measurement result, more evaluation results of HRO are provided based on several hardware platforms including Xilinx ZCU102, VCU110 and VCU118. The evaluation results present that specific HRO values are different, the trend of HRO remained unchanged with the change of CE scale. Therefore, the evaluation results can be generalized to other CNN models and hardware platforms.

IV. DESIGN SPACE EXPLORATION FOR INPUT TILE SIZE

In this section, the optimal input tile size is evaluated and selected for the design of our reconfigurable CNN acceleration architecture. Considering that the IFM size shows great variety across different CNNs, the tile-based method has been widely adopted in current dense and sparse CNN acceleration works [12], [13], [14], [15], [16], [17], [30], [31], [34], [35]. However, there are few works try to take design space exploration to evaluate multiple input tile sizes and select the most suitable one. Actually, the applied tile size is a crucial parameter for the implemented hardware architecture. Specifically, in many previous works, an input tile can be operated by a single CE in only one clock cycle, therefore, the input tile size has a strong relevant to the scale of a CE. Consequently, as depicted in equation (1), if the input and output parallelism is determined, the input tile size can significantly impact the peak performance of the accelerator. Furthermore, the input tile size also has a great impact on the bit-width between the input buffer and CE array.

$$Perf_{peak} = Para_{input} \times Para_{output} \times Scale_{CE} \times 2 \times Fre \quad (1)$$

Therefore, in this research, first, we choose eight tile sizes as the candidates of the optimal input tile size, including 1×1 to

TABLE IV
THE CONCLUSION OF IFM SIZE IN CURRENT CNNs

CNN	IFM Size
ResNet-18	224×224 112×112 56×56 28×28 14×14 7×7
VGG16	224×224 112×112 56×56 28×28 14×14 7×7
GoogLeNet	224×224 112×112 56×56 28×28 14×14 7×7
MobileNetV2	224×224 112×112 56×56 28×28 14×14 7×7
ResNet-50	224×224 112×112 56×56 28×28 14×14 7×7
InceptionV4	299×299 149×149 147×147 73×73 71×71 35×35 17×17 8×8
YOLOV3	256×256 128×128 64×64 32×32 16×16 8×8
YOLOV3-tiny	416×416 208×208 104×104 52×52 26×26 13×13

64×64 . Simultaneously, ten convolution modes, which are applied in current CNNs are also selected for the following evaluation. Subsequently, multiple input tile sizes are evaluated based on the following three metrics.

(1) Standard deviation of calculations (SDC): SDC represents that the deviation between the maximum and minimum number of computations in different convolution modes for a certain input tile size. As discussed in section II, we have pointed that the key difference of the convolution modes lies in their computation density, which can determine the number of computations in each input tile. Furthermore, while deploying the various number of computations to the fixed-scale CE array, the DSP utilization becomes unstable, thereby causing utilization fluctuation. Consequently, a novel metric, which is named SDC, is proposed and used to represent the computational span of an input tile under different convolution modes.

(2) The number of unsupported convolution mode (UCM): this metric reflects that the number of convolution modes that cannot be supported by each input tile size. As exhibited in TABLE I, there are many convolution modes, including multiple filter size, stride and convolution types, are applied in current CNNs, therefore, if the selected input tile size is smaller than the filter size, the designed acceleration architecture may have difficulty in operating this convolution mode.

(3) The number of unsuitable IFM size (UIS): this metric reflects that the number of IFM sizes that are not suitable to the selected input tile size. It seems that the larger the input tile are, the less the UCM is. However, as for UIS, the larger input tile would introduce nonnegligible invalid elements. For example, as shown in TABLE IV, 7×7 IFM appears in many classical CNNs such as VGG16 and GoogLeNet, if the input tile size is set to 8, it would result in introducing 23.4% invalid elements. Therefore, inducing UIS as a metric in the design space exploration for the input tile size would facilitate to eliminate the invalid elements and unnecessary computations.

A. The Input Tile Size Evaluation Based on SDC

The computations of different convolution modes under different input tile sizes are listed in TABLE V. As for 1×1 and 2×2 input tiles, only two convolution modes (i.e., 1×1 convolution with 1-stride and 2-stride) are included, and this is because the tile size is too small to operate the other convolution modes. As for 4×4 input tile, more than half of convolution modes can be supported, leaving four convolution modes including 5×5 convolution with 1-stride, 7×7 convolution with 2-stride, 1×7 convolution and 7×1 convolution with 1-stride. It is found that the filters in these convolution modes can be split into multiple filter

TABLE V
THE NUMBER OF COMPUTATIONS UNDER DIFFERENT TILE SIZE AND SDC

Tile Size	Filter size										SDC
	1×1/1	1×1/2	3×1/1	1×3/1	3×3/1	3×3/2	5×5/1	7×1/1	1×7/1	7×7/2	
1×1	1	1	/	/	/	/	/	/	/	/	0
2×2	4	1	/	/	/	/	/	/	/	/	0
4×4	16	4	24	24	36	9	/	/	/	/	27
4×4*	16	4	24	24	36	9	16/4/1	16/12	16/12	16/12/9	27
8×8	64	16	144	144	324	81	400	112	112	196	260
16×16	256	64	672	672	1764	441	3600	1120	1120	4900	4644
32×32	1024	256	2880	2880	8100	2025	19600	5824	5824	33124	32100
64×64	4096	1024	11904	11904	34596	8649	90000	25984	25984	164836	160740

TABLE VI
THE UCM AND UIS UNDER DIFFERENT TILE SIZE

CNN	UCM	UIS	UCM+UIS
1×1	8	0	8
2×2	8	0	8
4×4	4	0	4
4×4*	0	0	0
8×8	0	2	2
16×16	0	5	5
32×32	0	10	10
64×64	0	15	15

blocks so that all of the convolution modes can be simply supported by 4×4 input tile. Consequently, 4×4 input tile considering the aforementioned splitting method for filters is named optimized 4×4 input tile and listed as “ $4 \times 4^*$ ” in TABLE V and subsequent descriptions. It should be noted that there are multiple numbers of computations exist while operating the above three convolution modes using optimized 4×4 input tile, and this is because the computations in optimized 4×4 input tiles corresponding to different filter blocks varies. Furthermore, SDC of the evaluated tile sizes is calculated and listed in TABLE V, revealing an exponential growth in SDC as the tile size increases.

B. The Input Tile Size Evaluation Based on UCM and UIS

Unlike SDC, UCM and UIS can effectively evaluate the compatibility of different tile sizes. Unluckily, the above two metrics have totally different preferences. Specifically, as for smaller tile sizes (e.g., 1×1 input tile), it is suitable for all of the IFM sizes without inducing invalid elements, thereby UIS is 0, however, they can only support very limited convolution modes, causing highest UCM. Conversely, larger tile sizes (e.g., 64×64 input tile) can accommodate all of the convolution modes but may introduce numerous invalid elements and unnecessary calculations while dealing with most IFM sizes, leading to the highest UIS but 0 UCM. For better evaluation, we propose using UCM+UIS as a new metric and present it in TABLE VI, the results demonstrate that as the tile size increases, UCM+UIS initially decreases and then increases, while using optimized 4×4 input tile, UCM+UIS is 0.

C. The Selection for the Optimal Tile Size

Comprehensively considering the evaluation based on SDC, UCM and UIS, $4 \times 4^*$ is selected as the optimal tile size, which

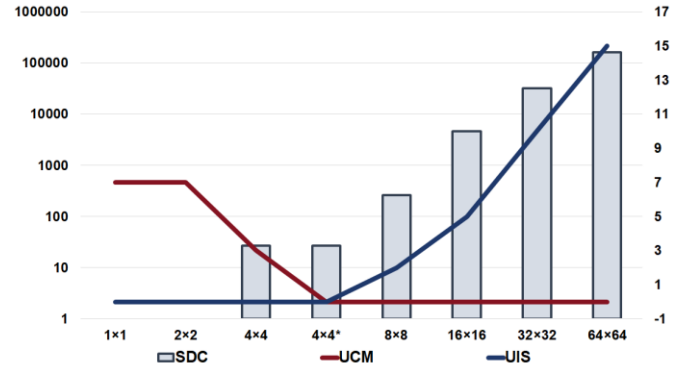


Fig. 3. The selection of input tile size.

is shown in Fig. 3. First, the evaluation based on SDC reveals that the input tile with smaller size would result in fewer SDC. Therefore, 1×1 is the optimal tile size if only considering SDC. However, when taking into account UCM and UIS, it become evident that $4 \times 4^*$ is the preferred choice since it can support all of the convolution modes while inducing neglectable invalid elements. Unluckily, there are two optimal tile sizes are selected by the above three metrics, we try to reexamine these two tile sizes (i.e., 1×1 and $4 \times 4^*$). Specifically, while considering UCM and UIS, 1×1 input tile can support all of the IFM sizes without inducing invalid elements, it lacks compatibility with most convolution modes. On the other hand, $4 \times 4^*$ input tile has the third smallest SDC while achieving the best UCM and UIS. Therefore, $4 \times 4^*$ is selected as the optimal input tile size.

V. DESIGN SPACE EXPLORATION FOR CE SCALE

After selecting the optimal input tile size. The following evaluation is for CE scale, which would impact the DSP utilization and hardware overhead of our proposed architecture. In this paper, a key idea is incorporating the design methodology of light-CE in sparse-based CNN acceleration into the implementation of reconfigurable CNN accelerator. Therefore, the candidates of CE scale are considered from 1 to 36, covering the maximum workload of a 4×4 input tile and the smallest CE scale applied in sparse-based acceleration works [15], [18], [19], [36]. Therefore, multiple CE scales are evaluated based on the following two metrics.

(1) DSP utilization rate (DUR): In section II, it is observed that a significant difference between these convolution modes is the computation density. The number of valid computations under different convolution modes would be various while using the given input tile size, thereby causing fluctuant

TABLE VII
THE DUR UNDER DIFFERENT CE SCALE

CE Scale	1×1/1	1×1/2	3×1/1	1×3/1	3×3/1	3×3/2	5×5/1	7×1/1	1×7/1	7×7/2
1	100%	100%	100%	100%	100%	100%	100%	100%	100%	100%
2	100%	100%	100%	100%	100%	90%	96%	100%	100%	98%
4	100%	100%	100%	100%	100%	75%	89%	100%	100%	94%
8	100%	50%	75%	75%	57%	89%	63%	88%	88%	88%
9	89%	44%	89%	89%	100%	100%	56%	78%	78%	78%
16	100%	25%	75%	75%	57%	56%	39%	88%	88%	77%
32	50%	13%	75%	75%	57%	28%	78%	44%	44%	38%
36	44%	11%	67%	67%	100%	25%	69%	39%	39%	34%

TABLE VIII
THE CALCULATION RATIO OF DIFFERENT CONVOLUTION MODES IN POPULAR CNNs

CNN	1×1/1	1×1/2	3×1/1	1×3/1	3×3/1	3×3/2	5×5/1	7×1/1	1×7/1	7×7/2
ResNet-18	0%	1.1%	0%	0%	82.9%	9.6%	0%	0%	0%	6.5%
VGG16	0%	0%	0%	0%	100%	0%	0%	0%	0%	0%
GoogleNet	23.3%	0%	0%	0%	61.0%	0%	8.2%	0%	0%	7.5%
MobileNetV2	85.9%	0%	0%	0%	3.3%	10.7%	0%	0%	0%	0%
ResNet-50	43.6%	7.6%	0%	0%	37.2%	8.6%	0%	0%	0%	2.9%
InceptionV4	27.0%	0%	2.2%	1.9%	21.6%	8.0%	0%	21.4%	17.9%	0%
YOLOV3	9.8%	0%	0%	0%	78.0%	12.1%	0%	0%	0%	0%
YOLOV3-tiny	4.2%	0%	0%	0%	95.8%	0%	0%	0%	0%	0%
Average	24.2%	1.1%	0.3%	0.2%	60.0%	6.1%	1.0%	2.7%	2.2%	2.1%

utilization rate of DSPs and even potentially impacting the overall performance of the accelerator. Hence, it is important to consider the DSP utilization rate (DUR) while evaluating the CE scale.

(2) Hardware resource overhead (HRO): Apart from DUR, the CE scale also significantly impacts the hardware resource overhead. Regarding the accumulations, increasing the scale of CE necessitates more accumulators inside CE while reducing the number of accumulators outside CE. Specifically, when the CE scale is set to one, which means one DSP is included in a single CE, only one serial adder inside CE is required to accumulate the single-products in different clock cycles. However, under this circumstance, the increasing input/output parallelism would induce more accumulators outside CE. On the contrary, if CE scale is set to 36, more parallel adders are needed inside PE but the decreasing input/output parallelism would induce fewer accumulators outside CE.

A. The CE Scale Evaluation Based on DUR

In this subsection, there are eight candidate CE scales are evaluated based on the aforementioned multiple convolution modes. As depicted in TABLE VII, it is evident that a smaller CE scale consistently leads to higher DUR. For instance, when CE scale is set to 1, the DUR for all of the convolution modes reaches 100%, if CE scale changes to 36, DUR decrease to less than 50% in over half of the convolution modes.

A straightforward evaluation method involves calculating the average DUR of these CE scales under different convolution modes. However, this method is unfair because all of the convolution modes are given the same weight in this method. Actually, these convolutional modes should be given different weights according to their computational proportions in CNNs. For example, 3×3 convolution with 1-stride is applied in most convolution layers of the above CNNs,

TABLE IX
AVERAGE CE UTILIZATION AND WEIGHTED CE UTILIZATION

CE Scale	Average CE Utilization	Weighted CE Utilization
1	100.0%	100.0%
2	98.4%	99.3%
4	95.8%	98.2%
8	77.3%	71.6%
9	80.1%	94.7%
16	68%	68.2%
32	50.2%	51.4%
36	49.5%	76.0%

whereas only InceptionV4 incorporates 1×7 convolution with 1-stride. Therefore, comparing 1×7 convolution with 1-stride, 3×3 convolution with 1-stride is more important and should thus be assigned a higher weight. TABLE VIII presents the calculation ratio of the above convolution modes in popular CNNs. Subsequently, the weight assigned to each convolution mode is determined as the average value of its calculation ratio of the listed CNNs. Based on these weights, TABLE IX illustrates both the average utilization and weighted utilization of CE. Compared with the average CE utilization, the actual application of each convolution mode is considered in the weighted CE utilization. It is found that with the increase of CE scale, the weighted CE utilization rate still shows a downward trend. However, two cases when the CE scale is 9 and 16 break this trend and achieve 94.7% and 76.0% weighted CE utilization, respectively.

B. The CE Scale Evaluation Based on HRO

1) *Multiplexers (MUXs) Inside CE*: First of all, the CE scale would impact the number of MUXs inside CE. Specifically,

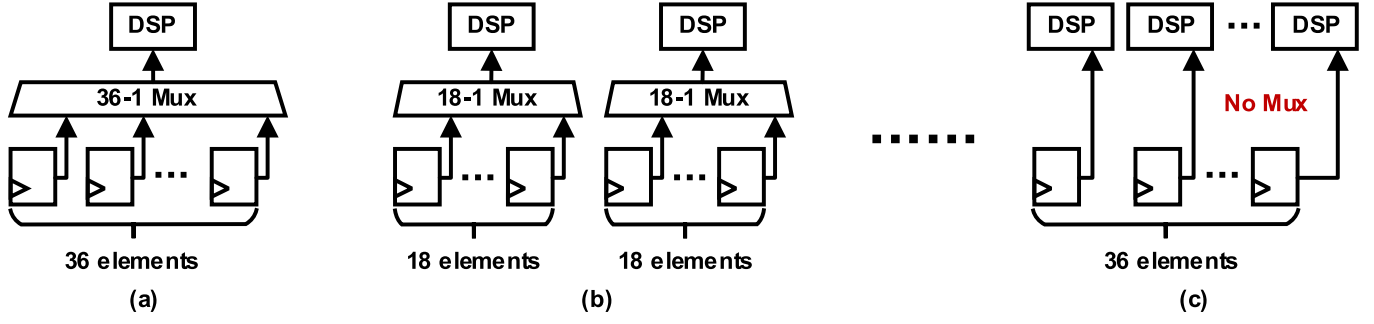


Fig. 4. The difference on multiplexers while applying different CE scale.

TABLE X
THE APPLICATION OF MUX UNDER DIFFERENT CE SCALE

CE Scale	Input/Output Parallelism	DSP Number	Mux Type	Mux Number in each CE	LUTs in each CE	Total LUTs
1	64/32	2048	36-1	2	22	45056
2	32/32	2048	18-1	4	24	24576
4	32/16	2048	9-1	8	32	16384
8	16/16	2048	5-1	16	32	8192
9	16/16	2304	4-1	18	18	4608
16	16/8	2048	3-1	32	32	4096
32	8/8	2048	2-1	64	64	4096
36	8/8	2304	/	/	0	0

if the CE scale is equivalent to the maximum workload, the weights and input pixels that stored in the registers can be directly transmitted to the following DSPs without utilizing MUXs, which is exhibited in Fig. 4(a). However, as the CE scale decreases, there will be more multiplications than available DSPs, and it is impossible to operate these multiplications within a clock cycle. This situation is illustrated in Fig. 4(b) and Fig. 4(c), where a pair of 18-1 MUXs or one 36-1 MUX is employed for selecting two or one weight and input pixel before transmitting them to the DSPs.

Therefore, we derive the hardware overhead for MUXs while using different CE scales in the hardware architecture and present them in TABLE X. It should be noted that the input/output parallelism changes with the variety of CE scale to maintain peak performance of the architecture. Consequently, as CE scale decreases, there is an increase in hardware cost due to a rise in input/output parallelism and MUX size.

2) *Accumulators Inside CE*: In addition to the MUXs inside CE, the CE scale also impacts the number of accumulators, thus leading to different HRO. Regarding the accumulators inside CE, which is presented in Fig. 5. Specifically, as shown in Fig. 5(a), when CE scale is set to 1, only one adder is required to accumulate the single-product (SP) from DSP and partial result (PR) to obtain the convolution result (CR) of one input channel. Moreover, if CE scale changes to 2 (Fig. 5(b)), two adders are employed within one CE to operate the parallel accumulation of two SPs and the serial accumulation of two PRs, respectively. Subsequently, when CE scale is set to the maximum value (i.e., 36), which is exhibited in Fig. 5(c), a total of 32 adders are utilized. Consequently, as demonstrated in TABLE XI, it is found that as the increasing of the CE scale, more SPs are generated, thereby the number of adders in one CE is increased and more hardware overhead is incurred. However, as depicted in TABLE XI, the hardware overhead

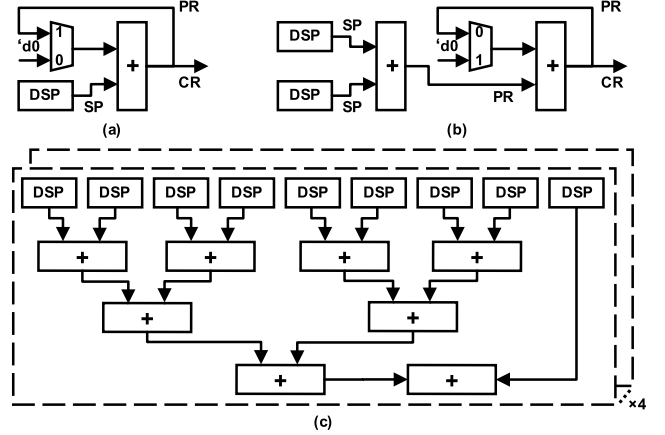


Fig. 5. The difference on accumulators inside CE while applying different CE scale.

TABLE XI
THE LUT CONSUMPTION FOR ADDERS UNDER DIFFERENT CE SCALE

CE Scale	Input/Output Parallelism	LUTs of Adders in One CE	LUTs of Adders inside CE	LUTs of Adders outside CE	Total LUTs of Adders
1	64/32	24	49152	803328	852740
2	32/32	40	40960	393842	434802
4	32/16	73	37376	197120	234496
8	16/16	140	35840	88832	124672
9	16/16	159	40704	88760	129464
16	16/8	251	32128	44416	76544
32	8/8	502	32128	22016	54144
36	8/8	1004	64256	21667	85293

caused by the accumulators inside CE of the whole CE array is also affected by the input/output parallelism. Therefore, this part of DUR does not show a linear alteration with the change of CE scale.

3) *Accumulators Outside CE*: Unlike the accumulators inside CE, the number of accumulators outside CE is solely impacted by the input/output parallelism. Specifically, taking example of the input parallelism, input parallelism reflects the number of input channels that operating multiplications in parallel. After operating multiplications and accumulations inside CE, the CRs should be accumulated again to compute the final convolution results. Consequently, the greater the input parallelism, the more adders are required to accumulate these CRs. Therefore, as shown in TABLE XI, this part of DUR increases as CE scale decreases.

TABLE XII
THE TOTAL LUT CONSUMPTION UNDER DIFFERENT CE SCALE

CE Scale	CE Utilization	CE Utilization ver.	Total LUT of Mux	Total LUT of Adders	Total LUT
1	100.0%	0.0%	45056	852740	897796
2	99.3%	0.7%	24576	434802	459378
4	98.2%	1.8%	16384	234496	250880
8	71.6%	28.4%	8192	124672	132864
9	94.7%	5.3%	4608	129464	134072
16	68.2%	31.8%	4096	76544	80640
32	51.4%	48.6%	4096	54144	58240
36	76.0%	24.0%	0	85293	85293

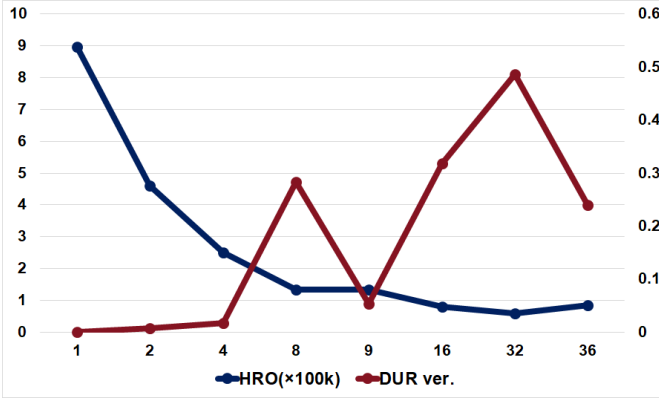


Fig. 6. The selection of CE scale.

C. The Selection for the Optimal CE Scale

After comprehensive evaluation based on DUR and HRO, which is shown in TABLE XII and Fig. 6, 9 is selected as the optimal CE scale. It should be noted that HRO is evaluated based on four hardware platforms, including Xilinx VC709, ZCU102, VCU110 and VCU118 FPGA platforms. The evaluation results present that while applying different FPGA platforms, although specific HRO values are different, the trend of HRO remains unchanged with the change of CE scale. Therefore, the evaluation results are still applicable for different hardware platforms. Specifically, TABLE XII presents the DSP utilization and LUTs for accumulators and MUXs. It should be noted that the inverse DSP utilization, calculated as $(1 - \text{DSP utilization}) \times 100\%$ is also provided in TABLE XII and applied in Fig. 6 for better evaluation and comparison. When compared to smaller CE scales (i.e., 1, 2, and 4), setting the CE scale to 9 exhibits similar DSP utilization but significant lower hardware overhead; whereas compared to larger CE scales (i.e., 16, 32, and 36), it demonstrates comparable hardware overhead but higher DSP utilization. Furthermore, when considering a CE scale of 8, significantly improved DSP utilization is achieved with almost identical hardware overhead.

VI. HARDWARE IMPLEMENTATION OF CE ARRAY AND ARCHITECTURE

In this section, our designed high-performance CE and reconfigurable CNN acceleration architecture are presented. First, based on the critical architectural parameters that selected in Section III and IV (i.e., input tile size and CE scale), a five-step CE is implemented for DSP utilization improvement and hardware resource reduction. Subsequently, with the aid of the above CE, we design a high-performance

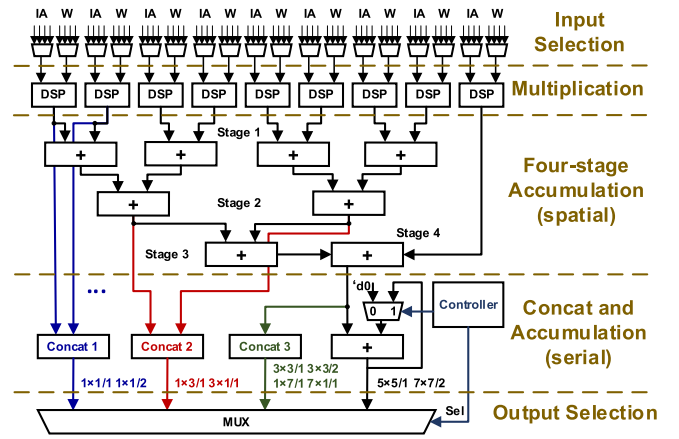


Fig. 7. The structure of our designed CE.

and reconfigurable CNN acceleration architecture, achieving great efficiency while deploying multiple convolution modes.

With the help of our selected architectural parameters, CE is designed to take multiplications and accumulations through a five-step workflow including input selection, multiplication, four-stage accumulation (spatial), concatenation and accumulation (serial) as well as the final output selection. As illustrated in Fig. 7, initially, the input tile size is set to 4×4 , implying that each input tile can accommodate a maximum of 36 multiplications. However, there are only 9 DSPs are placed in a CE. Consequently, 18 4-1 MUXs are applied to select nine weights and corresponding input pixels for subsequent processing by DSPs. Afterwards, in the second step, these multiplications are performed resulting in nine single-products. There are totally eight adders in the third step to form a four-stage adder tree, so that the aforementioned single-products can be accumulated to generate partial sums. It should be noted that different convolution modes always have various accumulation requirements. Therefore, a concatenation and serial accumulation step is set to support the above accumulation requirements. Specifically, as for 1×1 convolution, there is no need to take accumulations in CE, therefore, the single-products are directly sent to the concat 1 module (blue marked in Fig. 7). Additionally, only three single-products need to be accumulated for 1×3 or 3×1 convolutions, thus the accumulation results from stage 2 are sent to concat 2 module (red marked in Fig. 7). The remaining convolution modes (e.g., 3×3 or 1×7 convolution) require more single-product accumulations. Hence, the accumulation results from stage 4 are sent to concat 3 module (green marked in Fig. 7). Moreover, as for large size convolution (e.g., 7×7 convolution), its partial sums cannot be obtained in one clock cycle because of the limited DSPs in one CE. Therefore, an additional adder is employed to take serial accumulations for the single-products from different clock cycles. Finally, an output MUX is applied in the last step to select the partial sums according to the *Sel* signal from the controller that reflects the current convolution mode.

The proposed hardware architecture is exhibited in Fig. 8. Prior to launching the accelerator, the weights and IFMs are transferred from external memory (i.e., DDR) to the buffer component (i.e., in-out buffer and filter buffer) of the hardware accelerator. In the meanwhile, the configuration of CNN layers is also transmitted from CPU to the controller. Subsequently, execution commences on the proposed hardware, and the filters and 4×4 input tiles are submitted to the following

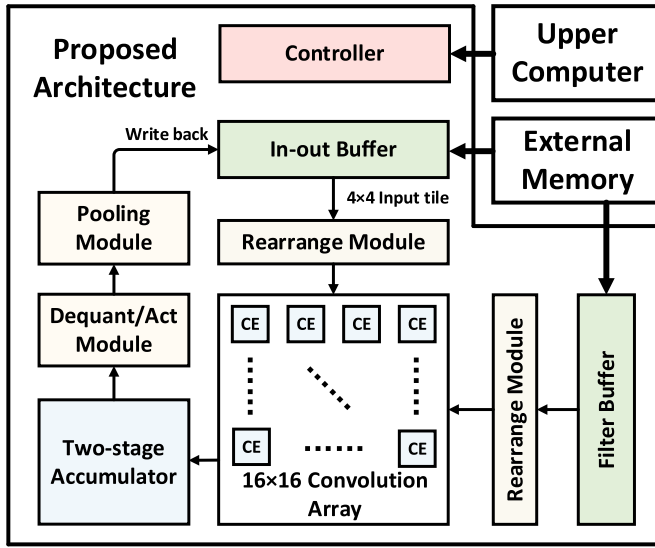


Fig. 8. Our proposed reconfigurable CNN architecture.

rearrange module, which is applied to operate data rearrangement. Afterwards, these reordered weights and input pixels are sent to the 16×16 convolution array consisting of multiple CEs to take convolutions, a two-stage accumulator is also applied to take accumulations for the partial sums from the convolution array to generate convolution results. The last two modules are used to handle the remaining operations, including quantization, activation and pooling to generate the final results that should be written back to the in-out buffer for subsequent layers' convolutions.

VII. HARDWARE EVALUATION AND COMPARISON

In this section, we first provide a detailed introduction to our experimental setting. Then, we evaluate the hardware overhead of our proposed hardware. Finally, regarding the hardware evaluation, first, an evaluation for CNN layers using different convolution modes is made based on our designed architecture to present its flexibility and reconfigurability. Furthermore, our architecture is also evaluated on some popular CNNs and compared with other current works to show its excellent performance.

A. Experimental Setup

Our experimental setup can be divided into two parts: experimental objects and experimental process. As for the experimental objects, first, a homemade CNN, which include eight modes of convolutions, is generated in this paper to evaluate the performance and DSP utilization of our designed hardware. Subsequently, five well-known CNNs, including VGG16, ResNet-18, ResNet-50, YOLOv3-tiny and YOLOv3 are deployed on our proposed architecture for further evaluations. And the main reason to apply these five CNNs is that they are widely-used in current works [16], [34], [37], [38], [40], [41], [42], the evaluations on the aforementioned CNNs can be used to take fair comparisons with these works. Furthermore, in terms of the experimental process, which includes hardware resource evaluation and performance evaluation. Our proposed architecture is implemented based on Xilinx VC709 FPGA platform. As for the test setup, the evaluation workflow is shown in Fig. 9. By using CPU, the configurations of CNN layers, well-trained CNNs and IFMs, are preprocessed to

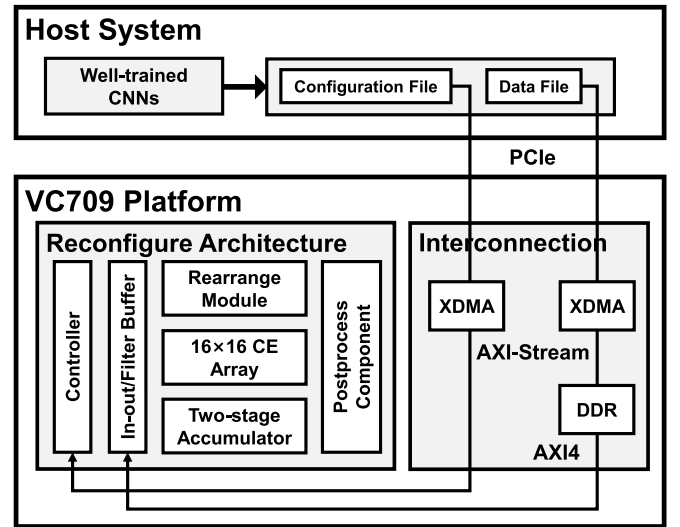


Fig. 9. Evaluation set up of our proposed architecture.

TABLE XIII
HARDWARE RESOURCE UTILIZATION OF OUR DESIGN

Resource	LUT	FF	DSP	BRAM
Our architecture	310439	405089	2432	909
Available	433200	866400	3600	1470
Ratio	71.67%	46.76%	67.56%	61.84%

TABLE XIV
HARDWARE RESOURCE UTILIZATION OF CE AND CE ARRAY

Resource	LUT	FF	DSP	BRAM
CE	343	834	9	0
CE array	87808	213504	2304	0
Ratio to the architecture	28.29%	52.71%	94.74%	0

generate binary files, including configuration file and data file. Specifically, the configuration file is applied to store the configuration information (e.g., filter size, the number of input/output channels) of each layer of CNNs, while the data file contains the weights of CNNs, bias, as well as IFMs. After obtaining the binary files, these files are submitted to the FPGA board through the Peripheral Component Interconnect express (i.e., PCIe). Consequently, the Direct Memory Access (i.e., DMA) controller sends the configuration file to the controller and delivers the well-trained CNNs and IFMs to double data rate (DDR) memory through the Advanced eXtensible Interface (AXI)-stream bus, respectively. Furthermore, the AXI4-full bus is adopted to interact between the DDR memory and on-chip buffer component (i.e., inout buffer and filter buffer).

B. Hardware Resource Evaluation

As shown in TABLE XIII, the number of used LUT and FF and the corresponding usage are 310439 (71.67%) and 405089 (46.76%), respectively. Compared with other important hardware resources (FF, DSP and BRAM), LUT utilization is the highest. And this is mainly because of the achievement for reconfigurability and high-performance. Specifically, taking the CE array, the most important module in our designed architecture, as an example. As shown in TABLE XIV, there are 343 LUTs are used in a CE. These LUTs are mainly

TABLE XV
THE RECONFIGURABILITY FOR CONVOLUTION PARAMETERS AND CNNs

Reconfigurability for convolution parameters		Reconfigurability for CNNs and corresponding scenarios	
Filter size	1×1-16×16 (2D convolution) 1×1-1×16 (1D convolution)	YOLOv3 YOLOv3-tiny YOLOv5 YOLOv8n	Object detection [48]
Stride	1-4	ResNet-50	Semantic segmentation [49]
IFM size	4-1024	MobileNetV1 MobileNetV2	Image classification [50]
Input channel	1-2048	U-Net	Lesion Segmentation [51]
Output channel	1-2048	InceptionV3	Gender classification [52]
Pooling size	1-16	VGG16	Garbage recognition and classification [53]

used to implement the aforementioned four-stage accumulators and accomplish selections from the accumulation results of different stages to provide a wide support for various convolution modes. Furthermore, the input/output parallelism of the architecture is 16/16, therefore, the high computation parallelism would improve the performance of our architecture with the increasing hardware overhead. Additionally, while achieving flexibility and high parallelism, the hardware resource utilization of our proposed architecture in VC709 is also balanced, and the utilization of all of the critical hardware resource (i.e., LUT, FF, DSP and BRAM) is lower than 75%. Therefore, with the aid of the balanced hardware utilization, our architecture can achieve an excellent runtime frequency (i.e., 300MHz).

C. Performance Evaluation and Comparisons

Firstly, the reconfigurability of the proposed architecture is introduced in detail. As shown in TABLE XV, our proposed architecture not only can support both of the 2D convolutions whose filter size ranges from 1×1 to 16×16 , but also can operate 1D convolutions whose filter size ranges from 1×1 to 1×16 . Therefore, multiple CNNs including YOLO series, ResNet series, MobileNet series and Inception series, which are employed for object detection, semantic segmentation, image classification and gender classification, respectively, can be deployed on this architecture.

Moreover, for the performance evaluation, first, an especial CNN, where hold eight convolution layers with different filter sizes and strides, is generated and deployed to our architecture. Considering that there is not a real CNN can include all of the convolution modes listed in TABLE I, thereby it is difficult to evaluate the compatibility to different convolution modes of our architecture through the evaluation for real CNNs. This is why we try to generate such a CNN whose convolution layers have different filter sizes and strides. As shown in Fig. 10, the performance and DSP utilization of each convolution layer are presented. It can be noted that our proposed architecture can realize excellent performance and DSP efficiency while operating different convolution layers.

Additionally, considering that the evaluation on home-made CNNs is difficult to compare with the current works that deploy generally-used CNNs, we try to deploy multiple classic CNNs, including VGG16, ResNet-18, ResNet-50, YOLOv3-tiny and YOLOv3, to our proposed architecture. Consequently, a horizontal comparison between our hardware and some pervious dense- and sparse- works are provided in TABLE XVI. There are totally eight related works [16], [34], [37], [38], [39], [40], [41], [42] are selected for the

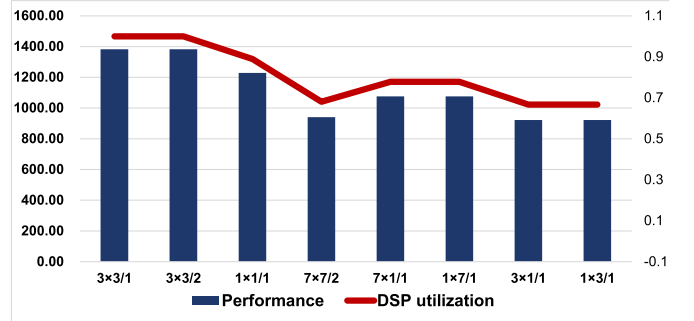


Fig. 10. Performance and DSP utilization for CNN layers with different convolution modes.

further evaluation. Considering that sparse-based methodology is employed to the design of our architecture, four sparse-based works [16], [34], [38], [40] are also chosen to see if the methodology can work well. As for VGG16, our proposed architecture can realize $1.35\times$ to $10.77\times$ improvements on performance while comparing to previous works [16], [34], [37], [38], [40], [41], [42]. Compared to works [38], [42], whose benchmark is ResNet-50, our architecture can also achieve $3.99\times$ to $5.70\times$ improvements on performance. Furthermore, based on another popular CNN YOLOv3-tiny, the performance of our hardware can reach 1128.5 GOPS, which is $9.01\times$ of that in work [34].

To make a fairer comparison, DSP efficiency, which is the ratio of the actual performance and the number of DSPs, is also evaluated and the comparison results are listed in TABLE XVI. Specifically, when VGG16 is deployed, our designed hardware can realize $1.22\times$ to $2.84\times$ improvements on DSP efficiency compared to related works [16], [34], [37], [40], [41], [42]. Additionally, our designed architecture can also realize $3.31\times$ and $2.00\times$ improvements on DSP efficiency with the deployment on ResNet-50 and YOLOv3-tiny, respectively. Moreover, it is observed that our DSP efficiency is lower than that of work [38], and this is mainly because of the ABM-SpConv method applied in work [38]. Specifically, work [38] claims that when the weights of CNNs are quantized in fixed-point format with q-bit precision, there only exists Q ($Q = 2^q$) unique quantization values. Therefore, the weight-reuse method of work [38] is that multiple input activations are multiplied with the weights which are quantified to the same value while executing convolutions, and the single-product should be accumulated to the same partial sum. Consequently, the input activations that should be multiplied with the same-value weight can be accumulated first and then multiplied with the weight, thus the required multiplications are greatly reduced. Furthermore, a low bit-width quantization and a weight clustering approach are also utilized in work [38] to generate more weight-reuse circumstances. Based on the utilization of weight-reuse, work [38] proposes an accumulation-multiplication method to effectively reduce the required multiplications and enhance the DSP efficiency. However, the performance of work [38] would be greatly affected by the convolution modes. Specifically, while operating the convolution mode with smaller filter size (e.g., 1×1 filter), the weight-reuse circumstance is greatly reduced. Consequently, while ResNet-50 is deployed on work [38], the actual performance is lower than that of ours. Additionally, as for DSP efficiency, there is no significant difference between work [38] and ours.

TABLE XVI
COMPARISONS WITH OTHER DESIGNS

	S. Kala [37] TVLSI 2019	D. Wang [38] TCAD 2020	A. Ahmad [39] TCAS-II 2021	X. Chang [40] TCAS-I 2021	Y. Liang [34] TCAD 2021	Y. Liang [16] TCAD 2022	X. Wu [41] TCAS-I 2022	D. Kim [42] TCAS-I 2023	Ours
CNN model	AlexNet/VGG16	VGG16-ResNet-50	ShuffleNet	VGG16	VGG16/Tiny-YOLO	AlexNet/VGG16/ResNet/GoogleNet	VGG16	VGG16/ResNet-50	VGG16/ResNet-18/ResNet-50/YOLOv3-tiny/YOLOv3
Accelerator type	dense	unstructured sparse	dense	structured sparse	unstructured sparse	unstructured sparse	dense	dense	dense
Platform	VC709	GXA7/GXC9	VC707	ZCU102	ZCU102	ZCU102	Arria 10	VCU118	VC709
Clocks (MHz)	200	200/101	200	300	166	200	200	200	300
Precision	Fixed 16	Fixed 8	Fixed 18	Fixed 8	Fixed 16	Fixed 16	Fixed 8	Fixed 8	Fixed 8
DSP	1436	256/342	1926	654	512	1144	607	2320	2432
LUT	468K	144K/100K	198K	72K	NA	552K	207K	814K	310K
FF	649K	NA	117K	58K	NA	NA	NA	795K	405K
BRAM	1465	2560/1120	1027	851	NA	912	769	1663	909
Power (W)	17.3	NA	9.3	23.7	23.6/9.6	23.6	NA	NA	16.2
Actual performance (GOPS)	433.6/407.2	1013/201	107.9	318.22	127.1/125.3	223.4/309.0/291.4/257.4	267.1	402/287.2	1369.5/991.4/146.2/1128.5/1296.7
DSP efficiency (GOPS/DSP)	0.302/0.284	3.957/0.588	0.056	0.487	0.248/0.245	0.218/0.302/0.285/0.251	0.440	0.209/0.150	0.594/0.430/0.497/0.490/0.562

TABLE XVII
COMPARISONS WITH SOTA ACCELERATORS ACROSS
DIFFERENT SCENARIOS

	J. Li [28] TCAS-II 2021	C. Chiu [46] TCAS-I 2022	J. Guam [47] TCSVT 2022	Ours
Application scenarios	Object detection	Face classification	Denoising	Object detection
Models	VGG16	eCNN	Deformable convolution	VGG16
Platform	VC709	ZCU102	ZCU102	VC709
Clocks (MHz)	200	100	250	300
Precision	Fixed 8/16	Fixed 8	Fixed 8	Fixed 8
DSP	664	1536	2304	2432
LUT	121K	88K	205K	310K
FF	160K	49K	294K	405K
BRAM	467	242	544	909
Actual performance (GOPS)	230.1	441.1	529.9	1369.5
DSP efficiency (GOPS/DSP)	0.347	0.287	0.230	0.594

Furthermore, the comparison with the reconfigurable accelerators for different scenarios, including object recognition [28], Chaos LiDAR based face classification [46] and ultra high definition (UHD) image denoising [47], is provided and presented in TABLE XVII. The comparison results show that our proposed architecture can realize $2.58\times$ to $5.95\times$ improvements on actual performance and $1.71\times$ to $2.58\times$ improvements on DSP efficiency. Therefore, our designed reconfigurable architecture can also realize advanced actual performance and DSP efficiency in the comparison across different scenarios.

The advantages of this work are emphasized. When compared to previous dense-based CNN accelerators [37], [39], [41], [42], our proposed architecture can achieve both of the reconfigurability and performance. In terms of the reconfigurability, limited convolution modes belonging to only one or two CNNs are supported by some works [37], [39]. Specifically, AlexNet and VGG16 are mapped on the work [37], while

only ShuffleNet is improved by another work [39]. Some other dense-based works [41], [42] can support a wider range of CNNs. However, the unified tile-based dataflow and high-utilization CE array are not evaluated or utilized in the above works for further performance enhancement. Various tile-based dataflow would induce more significant workload gap, and a CE scale without evaluation and selection can also cause unstable and inefficient DSP utilization. Therefore, in this study, the key difference between the proposed design and the prior dense-based designs lies in the sufficient evaluation for the critical architectural parameters (i.e., the tile size and CE scale). Specifically, the tile size is first evaluated based on design space exploration considering workload difference and suitability for IFMs and filters, moreover, the optimal CE scale is selected according to DSP utilization of different convolution modes and the hardware overhead. Therefore, with the aid of the design space exploration, the proposed architecture can achieve higher DSP utilization while operating different convolution modes, thereby achieving high performance and DSP efficiency while maintaining the reconfigurability.

Second, while comparing with sparse-based works [16], [34], [38], [40]. Our designed architecture can also realize better actual performance (actual computations/runtime), and this is because of the fine-grained structured sparse represented from different convolution modes. Specifically, multiple convolution modes with different filter sizes and strides are equivalent to the same convolution mode with different sparsities according to their computation densities. Consequently, the key difference of the proposed design and the prior sparse-based design is the improvement for the workload imbalance. Specifically, because the convolution mode and computational density remain constant within the same convolutional layer, the workload imbalance problem, which widely exists in the previous sparse-based works and significantly damages their performance, can be directly avoided in this work. Therefore, the actual performance and corresponding DSP efficiency of our work are greater than the aforementioned sparse-based CNN accelerators.

Furthermore, when compared to software-level optimizations [43], [44], [45], we claim that the software-level optimization can actually achieve the search for hardware parameters and mappings based on a wider range of DNN layers and the accurate estimation for latency and energy modeling. However, while comparing with the reconfigurable hardware implementation, the above works always target a specific hardware architecture, thereby the optimization effect may be limited by the fixed hardware structure and dataflow. Additionally, the stability of the software-level optimizations is also uncertain. Moreover, the above works concentrates more on the model establishment and optimization method, thereby lacking some hardware implementation details. Compared to the aforementioned works, although software optimization is not included in our work, the architectural parameter evaluation and hardware implementation are still based on a wide range of convolution modes and CNN layers. Therefore, our work can be seen as embedding software optimization into the underlying hardware architecture, and the mutual constraint between software optimization and hardware architecture is effectively avoided.

In terms of the power consumption, on the one hand, TABLE XVI presents the power consumption of this study and the previous works. And it is simply found that the power consumption of this work is competitive to most of previous works. On the other hand, it is observed that the utilization of BRAM is one of the main sources of power. Consequently, in this study, the utilization of BRAM is reduced for the enhancement of power consumption. Additionally, in current FPGA-based CNN accelerators, the commonly used index for evaluating area efficiency is DSP efficiency (i.e., Actual performance/DSP amount). This is because that DSPs are the key computation components and the main overhead of hardware resource. And the comparison results in TABLE XVI show that the DSP efficiency of our proposed architecture is better than that of the listed previous works. The excellent DSP efficiency achieved in this work mainly comes from the following two reasons: first, light-CEs are utilized in our designed architecture to flexibly support the multiple convolution modes with various computation density by serially operating them in different clock cycles. Furthermore, the DSP utilization of multiple convolution modes is also considered through the design space exploration to ensure the selected CE scale can realize the highest hardware utilization.

VIII. CONCLUSION

In this paper, a sparse-based design methodology is applied for the implementation of the reconfigurable CNN acceleration. Subsequently, two critical design parameters, including input tile size and CE scale, are evaluated based on SDC, UCM and UIS, DUR as well as HRO, respectively. With the aid of the optimal parameters, a high-parallelism and flexible CE array and a high-performance and reconfigurable CNN architecture are designed. The accelerator is implemented on a Xilinx VC709 FPGA and run at a clock frequency of 300 MHz, achieving 921.60 to 1382.40 GOPS while operating various convolution modes. Compared with previous dense-/sparse-based works, the proposed accelerator can realize $1.35\times$ to $10.77\times$ improvements on performance and $1.22\times$ to $2.84\times$ improvements on DSP efficiency while deploying VGG16.

REFERENCES

- [1] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Dec. 2015, pp. 1026–1034.
- [2] J. Hong, B. Sapp, and J. Philbin, "Rules of the road: Predicting driving behavior with a convolutional model of semantic interactions," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 8446–8454.
- [3] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2014, pp. 580–587.
- [4] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, New York, NY, USA, 2012, pp. 1097–1105.
- [5] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.
- [6] C. Szegedy et al., "Going deeper with convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Boston, MA, USA, Jun. 2015, pp. 1–9.
- [7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
- [8] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. A. Alemi, "Inception-v4, inception-ResNet and the impact of residual connections on learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 1, Feb. 2017, p. 31.
- [9] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Salt Lake City, UT, USA, Jun. 2018, pp. 4510–4520.
- [10] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," 2018, *arXiv:1804.02767*.
- [11] P. Adarsh, P. Rath, and M. Kumar, "YOLO v3-tiny: Object detection and recognition using one stage improved model," in *Proc. 6th Int. Conf. Adv. Comput. Commun. Syst. (ICACCS)*, Coimbatore, India, Mar. 2020, pp. 687–694.
- [12] C. Yang, Y. Wang, X. Wang, and L. Geng, "WRA: A 2.2-to-6.3 TOPS highly unified dynamically reconfigurable accelerator using a novel Winograd decomposition algorithm for convolutional neural networks," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 66, no. 9, pp. 3480–3493, Sep. 2019.
- [13] C. Yang, Y. Wang, X. Wang, and L. Geng, "A stride-based convolution decomposition method to stretch CNN acceleration algorithms for efficient and flexible hardware implementation," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 67, no. 9, pp. 3007–3020, Sep. 2020.
- [14] Z. Song et al., "DRQ: Dynamic region-based quantization for deep neural network acceleration," in *Proc. ACM/IEEE 47th Annu. Int. Symp. Comput. Archit. (ISCA)*, Valencia, Spain, May 2020, pp. 1010–1021.
- [15] Y. Meng, C. Yang, S. Xiang, J. Wang, K. Mei, and L. Geng, "An efficient CNN accelerator achieving high PE utilization using a dense-/sparse-aware redundancy reduction method and data-index decoupling workflow," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 31, no. 10, pp. 1537–1550, Oct. 2023.
- [16] Y. Liang et al., "An efficient hardware design for accelerating sparse CNNs with NAS-based models," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 3, pp. 597–613, Mar. 2022.
- [17] J. Yezep and S.-B. Ko, "Stride 2 1-D, 2-D, and 3-D Winograd for convolutional neural networks," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 28, no. 4, pp. 853–863, Apr. 2020.
- [18] C. Deng, Y. Sui, S. Liao, X. Qian, and B. Yuan, "GOSPA: An energy-efficient high-performance globally optimized sparse convolutional neural network accelerator," in *Proc. ACM/IEEE Annu. Int. Symp. Comput. Archit.*, Valencia, Spain, Jun. 2021, pp. 1110–1123.
- [19] A. Gondimalla, N. Chesnut, M. Thottethodi, and T. N. Vijaykumar, "SparTen: A sparse tensor accelerator for convolutional neural networks," in *Proc. 52nd Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Oct. 2019, pp. 151–165.
- [20] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
- [21] S. Han, J. Pool, J. Tran, and W. J. Dally, "Learning both weights and connections for efficient neural networks," 2015, *arXiv:1506.02626*.

- [22] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," 2015, *arXiv:1510.00149*.
- [23] T. Zhang et al., "A systematic DNN weight pruning framework using alternating direction method of multipliers," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 184–199.
- [24] W. Wen, C. Wu, Y. Wang, Y. Chen, and H. Li, "Learning structured sparsity in deep neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 2074–2082.
- [25] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang, "Learning efficient convolutional networks through network slimming," in *Proc. IEEE Int. Conf. Comput. Vis.*, Venice, Italy, Oct. 2017, pp. 2755–2763.
- [26] J. Luo, J. Wu, and W. Lin, "ThiNet: A filter level pruning method for deep neural network compression," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Venice, Italy, Oct. 2017, pp. 5068–5076.
- [27] H. Irmak, D. Ziener, and N. Alachiotis, "Increasing flexibility of FPGA-based CNN accelerators with dynamic partial reconfiguration," in *Proc. 31st Int. Conf. Field-Programmable Log. Appl. (FPL)*, Dresden, Germany, Aug. 2021, pp. 306–311.
- [28] J. Li, K.-F. Un, W.-H. Yu, P.-I. Mak, and R. P. Martins, "An FPGA-based energy-efficient reconfigurable convolutional neural network accelerator for object recognition applications," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 68, no. 9, pp. 3143–3147, Sep. 2021.
- [29] B. Huang et al., "IECA: An in-execution configuration CNN accelerator with 30.55 GOPS/mm² area efficiency," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 11, pp. 4672–4685, Nov. 2021.
- [30] Y. Liang, L. Lu, Q. Xiao, and S. Yan, "Evaluating fast algorithms for convolutional neural networks on FPGAs," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 4, pp. 857–870, Apr. 2020.
- [31] T. Abtahi, C. Shea, A. Kulkarni, and T. Mohsenin, "Accelerating convolutional neural network with FFT on embedded hardware," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 26, no. 9, pp. 1737–1749, Sep. 2018.
- [32] C. Yang, J. Wu, S. Xiang, L. Liang, and L. Geng, "A high-throughput and flexible architecture based on a reconfigurable mixed-radix FFT with twiddle factor compression and conflict-free access," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 31, no. 10, pp. 1472–1485, Oct. 2023.
- [33] P. Wang et al., "Understanding convolution for semantic segmentation," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, Lake Tahoe, NV, USA, Mar. 2018, pp. 1451–1460.
- [34] Y. Liang, L. Lu, and J. Xie, "OMNI: A framework for integrating hardware and software optimizations for sparse CNNs," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 40, no. 8, pp. 1648–1661, Aug. 2021.
- [35] W. Sun, D. Liu, Z. Zou, W. Sun, S. Chen, and Y. Kang, "Sense: Model-hardware codesign for accelerating sparse CNNs on systolic arrays," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 31, no. 4, pp. 470–483, Apr. 2023.
- [36] C. Yang, Y. Meng, J. Xi, S. Xiang, J. Wang, and K. Mei, "WRA-SS: A high-performance accelerator integrating Winograd with structured sparsity for convolutional neural networks," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 32, no. 1, pp. 164–177, Jan. 2024.
- [37] S. Kala, B. R. Jose, J. Mathew, and S. Nallesh, "High-performance CNN accelerator on FPGA using unified Winograd-GEMM architecture," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 12, pp. 2816–2828, Dec. 2019.
- [38] D. Wang, K. Xu, J. Guo, and S. Ghiasi, "DSP-efficient hardware acceleration of convolutional neural network inference on FPGAs," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 12, pp. 4867–4880, Dec. 2020.
- [39] A. Ahmad and M. A. Pasha, "Optimizing hardware accelerated general matrix-matrix multiplication for CNNs on FPGAs," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 67, no. 11, pp. 2692–2696, Nov. 2020.
- [40] X. Chang, H. Pan, W. Lin, and H. Gao, "A mixed-pruning based framework for embedded convolutional neural network acceleration," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 4, pp. 1706–1715, Apr. 2021.
- [41] X. Wu, Y. Ma, M. Wang, and Z. Wang, "A flexible and efficient FPGA accelerator for various large-scale and lightweight CNNs," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 3, pp. 1185–1198, Mar. 2022.
- [42] D. Kim, S. Jeong, and J.-Y. Kim, "Agamoto: A performance optimization framework for CNN accelerator with row stationary dataflow," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 6, pp. 2487–2496, Jun. 2023.
- [43] Y. Lin, M. Yang, and S. Han, "NAAS: Neural accelerator architecture search," in *Proc. 58th ACM/IEEE Design Autom. Conf. (DAC)*, San Francisco, CA, USA, Oct. 2021, pp. 1051–1056.
- [44] L. Mei, P. Houshmand, V. Jain, S. Giraldo, and M. Verhelst, "ZigZag: Enlarging joint architecture-mapping design space exploration for DNN accelerators," *IEEE Trans. Comput.*, vol. 70, no. 8, pp. 1160–1174, Aug. 2021.
- [45] C. Hong, Q. Huang, G. Dinh, M. Subedar, and Y. S. Shao, "DOSA: Differentiable model-based one-loop search for DNN accelerators," in *Proc. 56th Annu. IEEE/ACM Int. Symp. Microarchitecture*, Toronto, ON, Canada, Oct. 2023, pp. 209–224.
- [46] C.-T. Chiu et al., "Chaos LiDAR based RGB-D face classification system with embedded CNN accelerator on FPGAs," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 12, pp. 4847–4859, Dec. 2022.
- [47] J. Guan et al., "Memory-efficient deformable convolution based joint denoising and demosaicing for UHD images," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 32, no. 11, pp. 7346–7358, Nov. 2022.
- [48] M. Kim, K. Oh, Y. Cho, H. Seo, X. T. Nguyen, and H.-J. Lee, "A low-latency FPGA accelerator for YOLOv3-tiny with flexible layerwise mapping and dataflow," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 71, no. 3, pp. 1158–1171, Mar. 2024.
- [49] Y. Heryadi, E. Irwansyah, E. Miranda, H. Soeparno, Herlawati, and K. Hashimoto, "The effect of ResNet model as feature extractor network to performance of DeepLabV3 model for semantic satellite image segmentation," in *Proc. IEEE Asia-Pacific Conf. Geosci., Electron. Remote Sens. Technol. (AGERS)*, Jakarta, Indonesia, Dec. 2020, pp. 74–77.
- [50] L. Mpova, T. Shongwe, and A. Hasan, "Classification and detection of cyanosis images on lightly and darkly pigmented individual human skins using a fine-tuned MobileNet architecture," in *Proc. Int. Conf. Artif. Intell., Big Data, Comput. Data Commun. Syst. (icABCD)*, Durban, South Africa, Aug. 2023, pp. 1–5.
- [51] Y. Li, E. Chouzenoux, B. Charmettant, B. Benatsou, J.-P. Lamarque, and N. Lassau, "Lightweight U-Net for lesion segmentation in ultrasound images," in *Proc. IEEE 18th Int. Symp. Biomed. Imag. (ISBI)*, Nice, France, Apr. 2021, pp. 611–615.
- [52] R. Thangaraj, P. Pandiyan, T. Pavithra, V. K. Manavalasundaram, R. Sivaramakrishnan, and V. K. Kaliappan, "Deep learning based real-time face detection and gender classification using OpenCV and inception v3," in *Proc. Int. Conf. Advancements Electr., Electron., Commun., Comput. Autom. (ICAECA)*, Oct. 2021, pp. 1–6.
- [53] H. Wang, "Garbage recognition and classification system based on convolutional neural network VGG16," in *Proc. 3rd Int. Conf. Adv. Electron. Mater., Comput. Softw. Eng. (AEMCSE)*, Shenzhen, China, Apr. 2020, pp. 252–255.
- [54] Y.-H. Chen, T. Krishna, J. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, Jan. 2017.
- [55] Z. Yuan et al., "STICKER: An energy-efficient multi-sparsity compatible accelerator for convolutional neural networks in 65-nm CMOS," *IEEE J. Solid-State Circuits*, vol. 55, no. 2, pp. 465–477, Feb. 2020.